DystopAI Subscription, OAuth, and Authorization Launch Guide

A practical first implementation plan for selling the Electron/web app with Google login, Apple login, Stripe subscriptions, and server-side entitlement checks.

Bottom line

For the first saleable version, ship direct-download desktop plus hosted web account pages. Use Clerk for Google/Apple/email login, Stripe Billing for subscriptions, and your own backend as the only authority for app access. The desktop app should only ask your backend whether the signed-in user is currently entitled.

1. What To Do First

1. Pick the first sales channel: direct-download desktop plus web account portal. Avoid iOS App Store and Google Play until the account and entitlement system works.
2. Buy or finalize the production domain, for example dystopai.com, and set DNS through Cloudflare or another DNS provider.
3. Create production accounts for Clerk, Stripe, a Postgres database, and hosting. Add Apple Developer and Google Cloud once you configure Apple/Google login.
4. Replace the current dev-token gate in this repo with real hosted account login and a server entitlement endpoint.
5. Create Stripe products/prices and configure Checkout, Customer Portal, webhooks, and entitlements.
6. Deploy the backend first, then wire the Electron app to open browser login, poll for completion, store the resulting app session securely, and check entitlements at launch.
7. Run subscription-state tests before launch: active, trialing, past_due, canceled, unpaid, refunded, webhook retry, and offline grace.

2. Recommended V1 Stack

Layer	Recommended choice	Why
Identity	Clerk	Fast Google, Apple, email, passkeys, session management, production OAuth setup, and webhooks.
Billing	Stripe Billing	Best flexibility for subscriptions, Checkout, customer

		portal, webhooks, entitlements, coupons, invoices, and B2B expansion.
Database	Postgres via Supabase or Neon	Simple hosted relational store for users, Stripe customer IDs, subscription state, devices, and entitlement cache.
Backend	Your existing Express/TypeScript server, deployed separately	The current app already has Express. Keep the trusted subscription checks server-side.
Desktop auth	External browser login plus device-style polling	Works cleanly for Electron without embedding OAuth pages or putting provider secrets in the app.
Subscription management	Stripe Customer Portal	Users can update cards, change plans, download invoices, and cancel without you building billing UI.

When to choose Paddle or Lemon Squeezy instead

Choose Paddle or Lemon Squeezy if tax/VAT, merchant-of-record handling, and built-in customer billing management matter more than Stripe-level flexibility. Keep the same app architecture: auth provider plus billing webhooks plus your own entitlement API.

3. Accounts To Create

Account	Use it for	First setup tasks
Domain/DNS	Production URLs and OAuth redirect domains	Own domain, add DNS records, create app.dystopai.com and api.dystopai.com.
Clerk	User accounts and social login	Create production instance, enable Google/Apple/email, set allowed redirect URLs, create webhook signing secret.
Google Cloud	Google OAuth credentials	Create OAuth consent screen, web OAuth client for Clerk, configure authorized domains, publish or verify if scopes require it.
Apple Developer	Sign in with Apple and future macOS/iOS distribution	Register App ID, Services ID, Sign in with Apple key, domain/return URLs.
Stripe	Subscriptions and billing portal	Activate account, create products/prices, customer portal, webhook endpoint, entitlements, test mode

		first.
Database	Subscription truth cache	Create Postgres project, run schema, keep service credentials only on backend.
Hosting	Backend and web account portal	Deploy server/API, set HTTPS, set environment variables, add logs and health check.
Transactional email	Receipts beyond Stripe, support, security notices	Use Resend/Postmark/SES later; Stripe handles payment receipts to start.
Code signing	User trust for installers	Windows Authenticode or Azure Trusted Signing; Apple Developer ID/notarization if shipping macOS.

4. Architecture

The desktop app is not the authority. Your backend is. Stripe tells your backend what was paid. Clerk tells your backend who the user is. The app receives only a limited app session and an entitlement response.

```
User -> Electron app
Electron app -> POST /api/desktop-auth/start
Backend -> returns browser login URL + one-time desktop session id
Electron app -> opens system browser
Browser -> Clerk login with Google / Apple / email
Web account page -> marks desktop session approved
Electron app -> polls /api/desktop-auth/poll/:id
Backend -> returns app session token
Electron app -> GET /api/me/entitlements
Backend -> reads DB/Stripe cache and returns allowed features
Electron app -> unlocks only features listed by backend
```

5. What Code You Need To Write

Minimum production code for V1:

- Auth/session backend routes for desktop login start, browser approval, polling, refresh, and logout.
- Stripe Checkout route to start a subscription.
- Stripe Customer Portal route for plan/payment management.
- Stripe webhook route that verifies webhook signatures and updates your DB.
- Entitlement route that returns the current plan/features to the app.
- Frontend auth state replacing the current local dev token gate.
- Electron secure token storage and app-launch entitlement check.
- Optional device registration if one subscription should only activate a limited number of machines.

6. Database Schema

Use Postgres. Store provider IDs and Stripe IDs, but never store raw card data or OAuth provider secrets.

```
create table app_users (
  id uuid primary key default gen_random_uuid(),
  clerk_user_id text unique not null,
  email text not null,
  stripe_customer_id text unique,
  created_at timestamptz not null default now()
);

create table subscriptions (
  id uuid primary key default gen_random_uuid(),
  user_id uuid not null references app_users(id) on delete cascade,
  stripe_subscription_id text unique not null,
  stripe_customer_id text not null,
  status text not null,
  price_id text,
  current_period_end timestamptz,
  cancel_at_period_end boolean not null default false,
  updated_at timestamptz not null default now()
);

create table entitlements (
  user_id uuid primary key references app_users(id) on delete cascade,
  plan text not null default 'free',
  features text[] not null default '{}',
  source text not null default 'stripe',
  expires_at timestamptz,
  updated_at timestamptz not null default now()
);

create table desktop_auth_sessions (
  id uuid primary key default gen_random_uuid(),
  code_hash text unique not null,
  clerk_user_id text,
  status text not null default 'pending',
  expires_at timestamptz not null,
  created_at timestamptz not null default now()
);

create table user_devices (
  id uuid primary key default gen_random_uuid(),
  user_id uuid not null references app_users(id) on delete cascade,
  device_fingerprint_hash text not null,
  device_name text,
  last_seen_at timestamptz not null default now(),
  revoked_at timestamptz,
  unique (user_id, device_fingerprint_hash)
);
```

7. Backend Route Contracts

Route	Purpose	Trust rule

POST /api/desktop-auth/start	Create one-time desktop login session and return browser login URL.	No user trust yet. Generate random code and expiry.
POST /api/desktop-auth/approve	Called by signed-in web page after Clerk auth.	Requires valid Clerk session on web.
GET /api/desktop-auth/poll/:id	Desktop polls until approved.	Return app token only once, then expire session.
POST /api/billing/checkout	Create Stripe Checkout subscription session.	Requires authenticated user.
POST /api/billing/portal	Create Stripe Customer Portal session.	Requires authenticated user with Stripe customer ID.
POST /api/stripe/webhook	Receive Stripe subscription and entitlement changes.	Must use raw body and verify Stripe signature.
GET /api/me/entitlements	Return current feature access for the app.	Requires app session. Compute from DB cache, reconcile if stale.

8. Starter Backend Skeleton

This is a skeleton, not a paste-and-launch production server. It shows the shape of the code you add to your existing Express backend.

```
import express from 'express'
import Stripe from 'stripe'
import { ClerkExpressRequireAuth } from '@clerk/clerk-sdk-node'

const app = express()
const stripe = new Stripe(process.env.STRIPE_SECRET_KEY!)

app.post(
  '/api/stripe/webhook',
  express.raw({ type: 'application/json' }),
  async (req, res) => {
    const sig = req.header('stripe-signature')
    let event: Stripe.Event

    try {
      event = stripe.webhooks.constructEvent(
        req.body,
        sig!,
        process.env.STRIPE_WEBHOOK_SECRET!,
      )
```

```
    } catch {
      return res.status(400).send('Invalid webhook signature')
    }

    if (event.type === 'checkout.session.completed') {
      // Find user from session metadata.clerkUserId.
      // Store stripe_customer_id and subscription id.
    }

    if (event.type === 'customer.subscription.updated' ||
        event.type === 'customer.subscription.deleted') {
      // Upsert subscription status and current_period_end.
      // Recompute entitlements for that user.
    }

    if (event.type === 'entitlements.active_entitlement_summary.updated') {
      // Map Stripe entitlement lookup_keys into your local feature list.
    }

    return res.json({ received: true })
  },
)

app.use(express.json())

app.post('/api/billing/checkout', ClerkExpressRequireAuth(), async (req, res) => {
  const clerkUserId = req.auth.userId
  const user = await findOrCreateUser(clerkUserId)

  const session = await stripe.checkout.sessions.create({
    mode: 'subscription',
    customer: user.stripe_customer_id ?? undefined,
    customer_email: user.stripe_customer_id ? undefined : user.email,
    line_items: [{ price: process.env.STRIPE_PRICE_PRO_MONTHLY!, quantity: 1 }],
    success_url: `${process.env.APP_URL}/billing/success`,
    cancel_url: `${process.env.APP_URL}/billing/cancel`,
    metadata: { clerkUserId },
  })

  res.json({ url: session.url })
})

app.post('/api/billing/portal', ClerkExpressRequireAuth(), async (req, res) => {
  const user = await getUserByClerkId(req.auth.userId)
  const portal = await stripe.billingPortal.sessions.create({
    customer: user.stripe_customer_id,
    return_url: `${process.env.APP_URL}/account`,
  })
  res.json({ url: portal.url })
})
```

9. Frontend Changes In This Repo

The current app has a local token gate in src/context/AuthContext.tsx and src/components/auth/LoginModal.tsx. That is fine for development, but it cannot protect a paid app.

- Replace the password/token modal with buttons: Continue with Google, Continue with Apple, Continue with Email.
- For Electron, button click calls /api/desktop-auth/start and opens the returned URL in the system browser.
- Show a waiting state while the app polls /api/desktop-auth/poll/:id.
- After success, store the app session in OS secure storage. Do not use plain localStorage for production desktop sessions.
- At app launch, call /api/me/entitlements. If plan is free or inactive, keep the app usable but lock premium actions.
- Add Manage subscription button that calls /api/billing/portal and opens the Stripe portal URL.

```
type EntitlementsResponse = {
  active: boolean
  plan: 'free' | 'pro' | 'team'
  features: string[]
  expiresAt?: string
  graceUntil?: string
}

async function requireEntitlement(feature: string) {
  const res = await fetch('/api/me/entitlements', {
    headers: { Authorization: `Bearer ${sessionToken}` },
  })
  const data = await res.json() as EntitlementsResponse
  return data.active && data.features.includes(feature)
}
```

10. Entitlement Rules

Plan	Features	Server behavior
Free	Basic local UI, account page, docs, purchase screen	No paid model/runtime features. Good for onboarding.
Pro	Premium agents, model providers, updates, limited devices	Return active true while subscription is active/trialing or inside grace window.
Team	Multiple users, shared billing, device seats, admin portal	Add organization-level entitlements later after individual Pro works.

Offline grace

Cache a signed entitlement token for 24 to 72 hours so a paying user can open the desktop app during a temporary API outage. Do not cache forever. Re-check as soon as the backend is reachable.

11. Stripe Setup Checklist

8. Create products: DystopAI Pro Monthly, DystopAI Pro Annual, optionally Team.
9. Create recurring prices and record the price IDs in backend environment variables.
10. Configure Customer Portal to allow payment method updates, invoice downloads, cancellation, and plan changes.
11. Create entitlement features in Stripe, such as premium_agents, cloud_sync, priority_runtime, team_seats.
12. Attach features to Stripe products.
13. Create webhook endpoint: https://api.dystopai.com/api/stripe/webhook.
14. Listen for checkout.session.completed, invoice.paid, invoice.payment_failed, customer.subscription.updated, customer.subscription.deleted, and entitlements.active_entitlement_summary.updated.
15. Use Stripe CLI in test mode to simulate renewal, payment failure, cancellation, and entitlement update events.

12. OAuth Setup Checklist

With Clerk, Google and Apple OAuth happen on the hosted web auth surface. Your desktop app should open a real external browser instead of embedding the provider login screen.

Provider	Account setup	Important detail
Google	Google Cloud OAuth consent screen plus web client credentials in Clerk.	Desktop apps cannot keep client secrets. If you directly use Google OAuth, use Authorization Code with PKCE.
Apple	Apple Developer App ID, Services ID, Sign in with Apple private key, domains, return URLs.	If an Apple-platform app uses social login for primary account auth, Apple requires an equivalent privacy-focused login option.
Email	Enable Clerk email/password, magic link, or passkeys.	A first-party email login is useful fallback when users do not want Google or Apple.

13. App Store And Play Store Rules

Direct desktop/web is simpler

If you sell the Windows/macOS/Linux desktop installer from your own website, Stripe/Paddle/Lemon Squeezy are normal choices. If you sell digital features inside iOS App Store or Google Play apps, you must handle each store's in-app purchase policy before launch.

- Apple App Store: subscriptions must deliver ongoing value and work across the user's devices where the app is available. In-app digital purchases generally trigger App Store purchase rules.
- Apple login: Apple guideline 4.8 requires apps using third-party/social login for the user's primary account to offer an equivalent privacy-focused login service, with listed exceptions.
- Google Play: Play-distributed apps requiring payment for in-app digital features, app functionality, subscriptions, or cloud software generally must use Google Play Billing unless a policy exception applies.
- Practical launch path: do direct desktop/web first. Add mobile store billing later only if you build real mobile apps.

14. Security Rules

- Never place Stripe secret keys, Clerk secret keys, Apple private keys, or database credentials in the Electron client.
- Never trust subscription state sent by the app. The backend computes it from webhooks and reconciliation.
- Verify every Stripe webhook signature against the raw request body.
- Use short-lived app access tokens plus refresh tokens. Revoke refresh tokens on logout, refund fraud, or device revocation.
- Hash device fingerprints. Do not store unnecessary hardware identifiers in plaintext.
- Use HTTPS only in production. Add CSP on the web auth/account portal.
- Add audit logs for subscription changes, device activation, account deletion, and admin actions.
- Treat Electron client checks as user experience gates, not strong anti-piracy. Keep valuable server-backed services behind backend authorization.

15. Launch Test Plan

Test	Expected result
New user signs in with Google	Clerk user exists, local app gets app session, user has free entitlement.
New user signs in with Apple	Private relay email works, account is linked, user can continue.
User buys Pro monthly	Stripe Checkout completes, webhook stores

	customer/subscription, app unlocks Pro.
Payment fails	Subscription becomes past_due; app shows warning and grace state.
Subscription cancels	At period end, entitlement is revoked and paid features lock.
Webhook delivery delayed	Nightly reconciliation detects mismatch and corrects DB.
Backend offline	Cached signed entitlement works only inside grace window.
Device limit exceeded	App shows device management path instead of silently failing.

16. First Seven Days Plan

Day	Work
1	Choose final stack, create Clerk/Stripe/database/hosting accounts, set production domain plan.
2	Create DB schema, build user sync from Clerk, deploy backend health endpoint.
3	Build Stripe Checkout, Customer Portal, webhook verification, and local subscription cache.
4	Replace dev token UI with desktop browser login and polling flow.
5	Add /api/me/entitlements, feature gates, offline signed entitlement cache.
6	Test all Stripe states with test cards and Stripe CLI. Add logging and error screens.
7	Prepare launch pages: pricing, terms, privacy, support, refund/cancel policy, installer download.

17. Environment Variables

```
APP_URL=https://app.dystopai.com
API_URL=https://api.dystopai.com
DATABASE_URL=postgres://...

CLERK_SECRET_KEY=sk_live_...
CLERK_WEBHOOK_SECRET=whsec_...

STRIPE_SECRET_KEY=sk_live_...
STRIPE_WEBHOOK_SECRET=whsec_...
STRIPE_PRICE_PRO_MONTHLY=price_...
STRIPE_PRICE_PRO_ANNUAL=price_...

APP_SESSION_JWT_SECRET=generate-a-long-random-secret
ENTITLEMENT_SIGNING_SECRET=generate-a-different-long-random-secret
```

18. Definition Of Done Before Selling

- A user can sign up with Google, Apple, or email.
- A user can subscribe from a hosted pricing/account page.
- A user can open the desktop app and see paid features unlocked only after active entitlement.
- A user can manage/cancel their subscription through a hosted portal.
- Your backend survives webhook retries and can reconcile with Stripe.
- Your privacy policy, terms, support contact, and account deletion path exist.
- Your production keys are not in the repo, app bundle, public logs, or installer.
- The direct-download installer is code signed or at least clearly identified while you complete code signing.

19. What I Would Build First In This Repo

16. Add a separate production backend deployment instead of relying only on the local Electron-bundled server.
17. Create new server modules: authDesktop.ts, billingStripe.ts, entitlementService.ts, deviceService.ts.
18. Replace src/context/AuthContext.tsx with session/entitlement state instead of control-center-token.
19. Replace src/components/auth/LoginModal.tsx with browser login actions and a waiting state.
20. Add a Settings/Billing surface with current plan, renewal date, device list, and Manage subscription button.
21. Keep existing provider API-key auth separate from app-account auth. Provider keys let the runtime call models; subscription auth controls whether the customer can use premium app features.

20. Final Notes

This guide is technical implementation guidance, not legal, tax, or app-review legal advice. Before public launch, confirm tax, privacy, refund, and platform policy obligations for the countries and stores where you sell.

The strongest commercial control is not a hidden local license check. It is a server-backed account, webhook-updated entitlements, secure session handling, and premium services that require backend authorization.

Source List

Official docs checked on June 4, 2026:

- Stripe subscription webhooks: https://docs.stripe.com/billing/subscriptions/webhooks
- Stripe entitlements: https://docs.stripe.com/billing/entitlements
- Stripe customer portal: https://docs.stripe.com/billing/subscriptions/customer-portal
- Google OAuth for desktop apps: https://developers.google.com/identity/protocols/oauth2/native-app
- Clerk social connections: https://clerk.com/docs/guides/configure/auth-strategies/social-connections/overview
- Clerk production deployment: https://clerk.com/docs/guides/development/deployment/production
- Apple App Review Guidelines: https://developer.apple.com/app-store/review/guidelines/
- Sign in with Apple for web: https://developer.apple.com/help/account/capabilities/configure-sign-in-with-apple-for-the-web
- Google Play Payments policy: https://support.google.com/googleplay/android-developer/answer/9858738
- Paddle customer portal: https://developer.paddle.com/concepts/sell/customer-portal/
- Lemon Squeezy customer portal: https://docs.lemonsqueezy.com/help/online-store/customer-portal
- Lemon Squeezy merchant of record: https://docs.lemonsqueezy.com/help/payments/merchant-of-record